

TPI Component Inventory

Version: 1.0
Framework: React (Mantine/MUI compatible)
Last Updated: 2025-01-XX

Component Catalog

1. PortalLayout

Purpose: Root layout wrapper for all TPI portals. Provides AppShell structure with sidebar, topbar, and content area.

Props:

```
interface PortalLayoutProps {
  portalName: string; // e.g., "Technology Executive Dashboard"
  sidebarItems: SidebarItem[];
  user: User;
  children: React.ReactNode;
  maxContentWidth?: number; // default: 1400
  showBreadcrumbs?: boolean; // default: true
}
```

Variants:

- Standard: Full sidebar + topbar
- Collapsed: Icon-only sidebar (mobile/tablet)
- Minimal: No sidebar (full-width content)

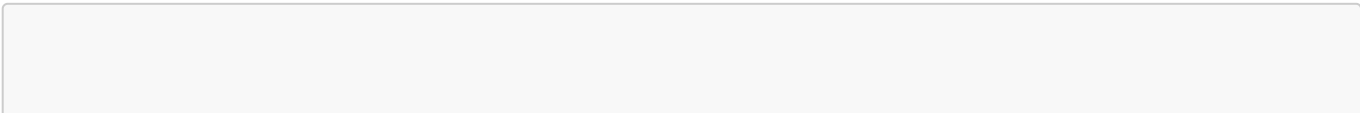
States:

- Loading: Skeleton sidebar + content
- Error: Error boundary with fallback UI
- Authenticated: Full layout
- Unauthenticated: Redirect to login

Accessibility:

- ARIA landmark: `role="main"` on content area
- Skip link: "Skip to main content"
- Keyboard: Tab navigation through sidebar

Example:



```
<PortalLayout
  portalName="Technology Executive Dashboard"
  sidebarItems={techExecSidebarItems}
  user={currentUser}
>
  <DashboardContent />
</PortalLayout>
```

2. RoleGuard

Purpose: Permission-based component wrapper. Conditionally renders children based on user role/permissions.

Props:

```
interface RoleGuardProps {
  requiredRole?: string | string[];
  requiredPermission?: string | string[];
  fallback?: React.ReactNode; // default: PermissionDenied component
  children: React.ReactNode;
}
```

Variants:

- Role-based: `requiredRole="admin"`
- Permission-based: `requiredPermission="incidents.view"`
- Multiple: `requiredRole=["admin", "cto"]` (OR logic)
- All: `requiredRole=["admin", "cto"] mode="all"` (AND logic)

States:

- Authorized: Renders children
- Unauthorized: Renders fallback (default: PermissionDenied)
- Loading: Shows spinner while checking permissions

Accessibility:

- Screen reader: Announces permission status
- Focus: Traps focus in fallback if unauthorized

Example:

```
<RoleGuard requiredPermission="incidents.create">
  <CreateIncidentButton />
</RoleGuard>
```

3. TopBar

Purpose: Global top navigation bar with search, actions, notifications, and user menu.

Props:

```
interface TopBarProps {  
  portalName: string;  
  onSearch?: (query: string) => void;  
  quickActions?: QuickAction[];  
  notifications?: Notification[];  
  user: User;  
  onUserMenuClick?: (action: string) => void;  
  showPortalSwitcher?: boolean;  
  availablePortals?: Portal[];  
}
```

Variants:

- Standard: Full feature set
- Minimal: Logo + user menu only
- Search-focused: Expanded search bar

States:

- Default: All features visible
- Search active: Search dropdown open
- Notification active: Notification panel open
- User menu open: Dropdown visible

Accessibility:

- Search: ARIA label "Global search"
- Keyboard: Cmd/Ctrl+K to open search
- Focus: Manage focus in dropdowns

Example:

```
<TopBar  
  portalName="Technology Executive Dashboard"  
  onSearch={handleGlobalSearch}  
  quickActions={quickActions}  
  notifications={notifications}  
  user={currentUser}  
>
```

4. SideNav

Purpose: Collapsible sidebar navigation with grouped items, icons, and permission-aware visibility.

Props:

```
interface SideNavProps {
  items: SidebarItem[];
  activePath: string;
  onNavigate: (path: string) => void;
  collapsed?: boolean;
  onCollapse?: (collapsed: boolean) => void;
}

interface SidebarItem {
  id: string;
  label: string;
  icon: React.ComponentType;
  path: string;
  badge?: string | number;
  children?: SidebarItem[];
  requiredRole?: string | string[];
  requiredPermission?: string | string[];
}
```

Variants:

- Expanded: Full labels + icons
- Collapsed: Icons only, tooltips on hover
- Mobile: Drawer overlay

States:

- Default: Expanded
- Collapsed: Icons only
- Active: Highlighted item with left border
- Hover: Subtle background highlight

Accessibility:

- ARIA: `aria-label` on navigation
- Keyboard: Arrow keys to navigate, Enter to select
- Focus: Visible focus indicator

Example:

```
<SideNav
  items={sidebarItems}
  activePath={currentPath}
  onNavigate={handleNavigate}
  collapsed={isCollapsed}
/>
```

5. Breadcrumbs

Purpose: Hierarchical navigation path showing current location and parent pages.

Props:

```
interface BreadcrumbsProps {
  items: BreadcrumbItem[];
  maxItems?: number; // default: 5, then ellipsis
}

interface BreadcrumbItem {
  label: string;
  path?: string; // if undefined, current page (non-clickable)
}
```

Variants:

- Standard: Full path visible
- Truncated: Ellipsis for long paths
- Mobile: Collapsed to "Back" button

States:

- Default: All items visible
- Truncated: "..." between first and last items
- Current: Last item bold, non-clickable

Accessibility:

- ARIA: `aria-label="Breadcrumb"`
- Keyboard: Tab through clickable items
- Screen reader: Announces current location

Example:

```
<Breadcrumbs
  items={[
    { label: "Dashboard", path: "/dashboard" },
    { label: "Incidents", path: "/incidents" },
    { label: "INC-2025-001" } // current page
  ]}
/>
```

6. PageHeader

Purpose: Standard page header with title, description, and action buttons.

Props:

```
interface PageHeaderProps {  
  title: string;  
  description?: string;  
  actions?: React.ReactNode;  
  badge?: string | number;  
  icon?: React.ComponentType;  
  backButton?: boolean;  
  onBack?: () => void;  
}
```

Variants:

- Standard: Title + description + actions
- Minimal: Title + actions only
- With badge: Title + badge indicator
- With icon: Title + icon prefix

States:

- Default: All elements visible
- Loading: Skeleton title + actions
- Error: Error state (optional)

Accessibility:

- Heading: **h1** for title (page-level heading)
- Description: **p** with descriptive text
- Actions: Keyboard accessible buttons

Example:

```
<PageHeader  
  title="Incident Management"  
  description="Track and resolve platform incidents"  
  actions={  
    <>  
      <Button variant="outline">Export</Button>  
      <Button>New Incident</Button>  
    </>  
  }  
  badge={5}  
</PageHeader>
```

Purpose: Key Performance Indicator display card with value, label, trend, and optional chart.

Props:

```
interface KpiCardProps {
  label: string;
  value: string | number;
  trend?: {
    value: number; // percentage change
    direction: "up" | "down" | "neutral";
    period: string; // e.g., "vs last week"
  };
  icon?: React.ComponentType;
  chart?: React.ReactNode; // mini sparkline chart
  format?: "number" | "currency" | "percentage" | "duration";
  loading?: boolean;
  error?: string;
}
```

Variants:

- Standard: Value + label + trend
- With chart: Value + mini sparkline
- With icon: Value + icon prefix
- Compact: Smaller padding for dense layouts

States:

- Default: All data visible
- Loading: Skeleton value + label
- Error: Error message display
- Empty: "No data" state

Accessibility:

- ARIA: `aria-label` with full context
- Screen reader: Announces value, label, trend

Example:

```
<KpiCard
  label="System Uptime"
  value={99.97}
  format="percentage"
  trend={{
    value: 0.02,
    direction: "up",
    period: "vs last month"
  }}
/>
```

```
    icon={TrendingUp}  
  />
```

8. StatusBadge

Purpose: Color-coded status indicator with semantic meaning (R/Y/G).

Props:

```
interface StatusBadgeProps {  
  status: "success" | "warning" | "error" | "info" | "neutral";  
  label: string;  
  size?: "sm" | "md" | "lg"; // default: "md"  
  variant?: "solid" | "outline" | "subtle"; // default: "solid"  
  icon?: React.ComponentType;  
}
```

Color Mapping:

- **success**: Green (#10b981)
- **warning**: Yellow (#f59e0b)
- **error**: Red (#ef4444)
- **info**: Blue (#3b82f6)
- **neutral**: Gray (#6b7280)

Variants:

- Solid: Colored background, white text
- Outline: Colored border, colored text
- Subtle: Colored background (10% opacity), colored text

States:

- Default: Normal display
- Hover: Slight scale (1.05x) if interactive
- Disabled: Reduced opacity

Accessibility:

- ARIA: **aria-label** with status context
- Color: Not sole indicator (icon or text also conveys status)

Example:

```
<StatusBadge  
  status="success"  
  label="Active"  
  size="sm"
```

```
    icon={CheckCircle}
  />
```

9. DataTable

Purpose: Enterprise-grade data table with sorting, filtering, pagination, selection, and export.

Props:

```
interface DataTableProps<T> {
  data: T[];
  columns: ColumnDef<T>[];
  loading?: boolean;
  pagination?: {
    page: number;
    pageSize: number;
    total: number;
    onPageChange: (page: number) => void;
    onPageSizeChange: (size: number) => void;
  };
  sorting?: {
    column: string;
    direction: "asc" | "desc";
    onSort: (column: string, direction: "asc" | "desc") => void;
  };
  filtering?: {
    filters: Filter[];
    onFilterChange: (filters: Filter[]) => void;
  };
  selection?: {
    selected: string[];
    onSelectionChange: (selected: string[]) => void;
    selectable?: (row: T) => boolean;
  };
  onRowClick?: (row: T) => void;
  exportable?: boolean;
  onExport?: (format: "csv" | "pdf") => void;
  emptyState?: React.ReactNode;
  errorState?: React.ReactNode;
  density?: "compact" | "standard" | "spacious";
}

interface ColumnDef<T> {
  id: string;
  header: string;
  accessor: (row: T) => any;
  sortable?: boolean;
  filterable?: boolean;
  render?: (value: any, row: T) => React.ReactNode;
  width?: number | string;
```

```
align?: "left" | "center" | "right";
}
```

Variants:

- Standard: Full feature set
- Minimal: No pagination/filtering (for small datasets)
- Read-only: No selection, no row click

States:

- Loading: Skeleton rows
- Empty: Empty state component
- Error: Error state component
- Default: Data displayed

Accessibility:

- ARIA: `role="table"`, `aria-sort` on sortable headers
- Keyboard: Arrow keys to navigate, Space to select, Enter to activate
- Screen reader: Announces row count, sort state

Example:

```
<DataTable
  data={incidents}
  columns={incidentColumns}
  pagination={{
    page: currentPage,
    pageSize: 25,
    total: totalIncidents,
    onPageChange: setPage,
    onPageSizeChange: setPageSize
  }}
  sorting={{
    column: "created_at",
    direction: "desc",
    onSort: handleSort
  }}
  onClick={row => openDrawer(row.id)}
  exportable
  onExport={handleExport}
/>
```

10. FilterBar

Purpose: Horizontal filter controls with chips for active filters and saved views.

Props:

```
interface FilterBarProps {
  filters: Filter[];
  activeFilters: Filter[];
  onFilterChange: (filters: Filter[]) => void;
  savedViews?: SavedView[];
  onSaveView?: (name: string, filters: Filter[]) => void;
  onLoadView?: (view: SavedView) => void;
  onDeleteView?: (viewId: string) => void;
}

interface Filter {
  id: string;
  type: "text" | "select" | "date" | "dateRange" | "boolean" | "number";
  label: string;
  value: any;
  options?: { label: string; value: any }[]; // for select
}

interface SavedView {
  id: string;
  name: string;
  filters: Filter[];
  isDefault?: boolean;
}
```

Variants:

- Standard: All filter types
- Compact: Collapsed filters (show active only)
- With saved views: Filter bar + saved views dropdown

States:

- Default: Filters visible
- Active: Active filter chips displayed
- Collapsed: "Filters" button, expandable panel

Accessibility:

- ARIA: `aria-label` on filter controls
- Keyboard: Tab through filters, Enter to apply

Example:

```
<FilterBar
  filters={availableFilters}
  activeFilters={currentFilters}
  onFilterChange={setFilters}
  savedViews={savedViews}
```

```
onSaveView={handleSaveView}  
/>
```

11. DetailDrawer

Purpose: Right-side slide-out drawer for viewing/editing entity details.

Props:

```
interface DetailDrawerProps {  
  open: boolean;  
  onClose: () => void;  
  title: string;  
  entityId?: string;  
  children: React.ReactNode;  
  width?: number | string; // default: 600px  
  footer?: React.ReactNode;  
  unsavedChanges?: boolean;  
  onSave?: () => void;  
  loading?: boolean;  
}
```

Variants:

- Standard: 600px width
- Wide: 800px width (for complex forms)
- Narrow: 400px width (for simple views)

States:

- Closed: Hidden (off-screen)
- Opening: Slide-in animation (300ms)
- Open: Visible, content loaded
- Closing: Slide-out animation (300ms)
- Unsaved: Warning on close attempt

Accessibility:

- ARIA: `role="dialog", aria-modal="true"`
- Focus: Trap focus in drawer when open
- Keyboard: Escape to close, Tab to navigate

Example:

```
<DetailDrawer  
  open={drawerOpen}  
  onClose={handleClose}  
  title="Incident Details"
```

```
    entityId={selectedIncidentId}
    width={700}
    footer={
      <Button onClick={handleSave}>Save Changes</Button>
    }
  >
  <IncidentDetails id={selectedIncidentId} />
</DetailDrawer>
```

12. ConfirmDialog

Purpose: Standard confirmation modal for destructive or important actions.

Props:

```
interface ConfirmDialogProps {
  open: boolean;
  onClose: () => void;
  onConfirm: () => void;
  title: string;
  message: string;
  confirmLabel?: string; // default: "Confirm"
  cancelLabel?: string; // default: "Cancel"
  variant?: "default" | "destructive"; // default: "default"
  requireTyping?: string; // e.g., "DELETE" for extra confirmation
  loading?: boolean;
}
```

Variants:

- Standard: Info confirmation
- Destructive: Red confirm button, warning icon
- Typed confirmation: Requires typing confirmation text

States:

- Closed: Hidden
- Open: Visible with backdrop
- Loading: Confirm button disabled, spinner
- Typed: Confirm button enabled (if requireTyping)

Accessibility:

- ARIA: `role="alertdialog"` for destructive
- Focus: Trap focus, initial focus on cancel
- Keyboard: Escape to cancel, Enter to confirm

Example:

```
<ConfirmDialog
  open={deleteDialogOpen}
  onClose={() => setDeleteDialogOpen(false)}
  onConfirm={handleDelete}
  title="Delete Incident"
  message="Are you sure you want to delete this incident? This action cannot be
  undone."
  variant="destructive"
  requireTyping="DELETE"
/>
```

13. EmptyState

Purpose: Standardized empty state display when no data is available.

Props:

```
interface EmptyStateProps {
  icon?: React.ComponentType;
  title: string;
  description: string;
  action?: {
    label: string;
    onClick: () => void;
  };
  illustration?: React.ReactNode;
}
```

Variants:

- Standard: Icon + title + description + action
- Minimal: Title + description only
- With illustration: Custom illustration instead of icon

States:

- Default: All elements visible
- Loading: Skeleton (if applicable)

Accessibility:

- ARIA: `role="status", aria-live="polite"`
- Screen reader: Announces empty state

Example:

```
<EmptyState
  icon={Inbox}
```

```
title="No incidents found"
description="Get started by creating your first incident report."
action={{
  label: "Create Incident",
  onClick: handleCreate
}}
/>
```

14. ErrorState

Purpose: Standardized error display for failed data loads or operations.

Props:

```
interface ErrorStateProps {
  title?: string; // default: "Something went wrong"
  message: string;
  error?: Error;
  retry?: {
    label: string;
    onRetry: () => void;
  };
  actions?: React.ReactNode;
}
```

Variants:

- Standard: Error icon + message + retry
- Minimal: Message only
- With details: Expandable error details

States:

- Default: Error visible
- Retrying: Retry button disabled, spinner

Accessibility:

- ARIA: `role="alert"`, `aria-live="assertive"`
- Screen reader: Announces error immediately

Example:

```
<ErrorState
  message="Failed to load incidents. Please check your connection and try again."
  retry={{
    label: "Retry",
    onRetry: handleRetry
```

```
  }}  
/>
```

15. SkeletonLoader

Purpose: Animated placeholder skeletons that match content structure.

Props:

```
interface SkeletonLoaderProps {  
  variant: "text" | "circular" | "rectangular" | "card" | "table" | "list";  
  width?: number | string;  
  height?: number | string;  
  count?: number; // for multiple skeletons  
  animation?: "pulse" | "wave" | "none"; // default: "pulse"  
}
```

Variants:

- Text: Line of text skeleton
- Circular: Circle skeleton (for avatars)
- Rectangular: Rectangle skeleton (for images)
- Card: Card structure skeleton
- Table: Table row skeleton
- List: List item skeleton

States:

- Loading: Animated shimmer
- Loaded: Replaced by actual content

Accessibility:

- ARIA: `aria-busy="true"`, `aria-label="Loading"`
- Screen reader: Announces loading state

Example:

```
{loading ? (  
  <SkeletonLoader variant="table" count={5} />  
) : (  
  <DataTable data={data} />  
)}
```

16. AuditTrail

Purpose: Display chronological change history for an entity.

Props:

```
interface AuditTrailProps {
  entityId: string;
  entityType: string;
  format?: "timeline" | "table"; // default: "timeline"
  limit?: number;
  exportable?: boolean;
}

interface AuditEntry {
  id: string;
  timestamp: string;
  user: {
    id: string;
    name: string;
    role: string;
  };
  action: string;
  changes?: {
    field: string;
    before: any;
    after: any;
  }[];
  reason?: string;
  ipAddress?: string;
}
```

Variants:

- Timeline: Vertical timeline with dates
- Table: Sortable table format
- Compact: Condensed view for lists

States:

- Loading: Skeleton timeline/table
- Empty: "No changes recorded"
- Default: Entries displayed

Accessibility:

- ARIA: `role="log"`, `aria-live="polite"`
- Keyboard: Navigate through entries

Example:

```
<AuditTrail
  entityId={incidentId}
  entityType="incident"
  format="timeline"
  exportable
/>
```

17. ActivityFeed

Purpose: Real-time activity stream showing recent actions and events.

Props:

```
interface ActivityFeedProps {
  activities: Activity[];
  loading?: boolean;
  onLoadMore?: () => void;
  hasMore?: boolean;
  realTime?: boolean; // enable real-time updates
}

interface Activity {
  id: string;
  timestamp: string;
  type: "action" | "event" | "system";
  actor?: {
    name: string;
    avatar?: string;
  };
  action: string;
  target?: {
    type: string;
    name: string;
    link?: string;
  };
  metadata?: Record<string, any>;
}
```

Variants:

- Standard: Full activity details
- Compact: Condensed view
- Real-time: Auto-updates with new activities

States:

- Loading: Skeleton activities
- Empty: "No recent activity"

- Default: Activities displayed

Accessibility:

- ARIA: `role="log", aria-live="polite"`
- Screen reader: Announces new activities

Example:

```
<ActivityFeed
  activities={recentActivities}
  realTime
  onLoadMore={loadMoreActivities}
  hasMore={hasMoreActivities}
/>
```

18. FormSection + FieldHelpText

Purpose: Grouped form sections with consistent spacing and help text.

FormSection Props:

```
interface FormSectionProps {
  title: string;
  description?: string;
  children: React.ReactNode;
  collapsible?: boolean;
  defaultCollapsed?: boolean;
}
```

FieldHelpText Props:

```
interface FieldHelpTextProps {
  text: string;
  variant?: "info" | "warning" | "error";
  icon?: React.ComponentType;
}
```

Variants:

- Standard: Title + description + fields
- Collapsible: Expandable/collapsible section
- With help: Fields with inline help text

States:

- Default: Expanded
- Collapsed: Hidden content
- Error: Error state in section

Accessibility:

- ARIA: `aria-describedby` linking fields to help text
- Screen reader: Reads help text when field focused

Example:

```
<FormSection
  title="Incident Details"
  description="Basic information about the incident"
>
  <Input
    label="Title"
    {...register("title")}
  />
  <FieldHelpText text="A brief, descriptive title for the incident" />
</FormSection>
```

19. Stepper

Purpose: Multi-step workflow indicator and navigation (for onboarding, release checklists).

Props:

```
interface StepperProps {
  steps: Step[];
  currentStep: number;
  onStepClick?: (step: number) => void;
  orientation?: "horizontal" | "vertical"; // default: "horizontal"
  variant?: "default" | "compact";
}

interface Step {
  id: string;
  label: string;
  description?: string;
  status: "pending" | "active" | "completed" | "error";
  optional?: boolean;
}
```

Variants:

- Horizontal: Steps in a row (desktop)
- Vertical: Steps stacked (mobile/compact)

- Compact: Smaller spacing, icons only

States:

- Pending: Gray, not clickable
- Active: Primary color, current step
- Completed: Green checkmark, clickable
- Error: Red, indicates failure

Accessibility:

- ARIA: `role="navigation", aria-label="Steps"`
- Keyboard: Arrow keys to navigate, Enter to select

Example:

```
<Stepper
  steps={releaseSteps}
  currentStep={currentStepIndex}
  onStepClick={handleStepClick}
/>
```

20. ExportButton

Purpose: Standardized export functionality with format selection and progress indication.

Props:

```
interface ExportButtonProps {
  onExport: (format: "csv" | "pdf" | "xlsx") => void;
  formats?: ("csv" | "pdf" | "xlsx")[]; // default: ["csv"]
  label?: string; // default: "Export"
  dataCount?: number; // shows "Export (247 items)"
  loading?: boolean;
  disabled?: boolean;
}
```

Variants:

- Standard: Button with dropdown
- Icon only: Icon button with tooltip
- With count: Shows item count

States:

- Default: Ready to export
- Loading: Spinner, disabled
- Success: Brief success state

- Error: Error tooltip

Accessibility:

- ARIA: `aria-label` with format context
- Keyboard: Space/Enter to open menu

Example:

```
<ExportButton
  onExport={handleExport}
  formats={['csv', 'pdf']}
  dataCount={filteredData.length}
/>
```

Component Usage Guidelines

When to Use Each Component

Layout Components:

- `PortalLayout`: Every portal page (root level)
- `PageHeader`: Every page (below breadcrumbs)
- `Breadcrumbs`: Every sub-page (except dashboard)

Data Display:

- `DataTable`: Lists of entities (>5 items)
- `KpiCard`: Dashboard metrics
- `StatusBadge`: Status indicators (tables, cards)

Navigation:

- `SideNav`: Portal navigation
- `TopBar`: Global navigation
- `Breadcrumbs`: Page hierarchy

Feedback:

- `EmptyState`: No data scenarios
- `ErrorState`: Error scenarios
- `SkeletonLoader`: Loading states
- `Toast`: Success/error notifications

Workflows:

- `DetailDrawer`: View/edit details
- `ConfirmDialog`: Confirmations
- `Stepper`: Multi-step processes

Component Composition Examples

Dashboard Page

```
<PortalLayout>
  <PageHeader title="Dashboard" />
  <div className="grid grid-cols-4 gap-4">
    <KpiCard label="Uptime" value={99.97} />
    <KpiCard label="Incidents" value={3} />
    <KpiCard label="Response Time" value="2.3s" />
    <KpiCard label="Active Users" value={1247} />
  </div>
  <ActivityFeed activities={activities} />
</PortalLayout>
```

List Page

```
<PortalLayout>
  <PageHeader
    title="Incidents"
    actions={<Button>New Incident</Button>}
  />
  <FilterBar filters={filters} />
  <DataTable
    data={incidents}
    columns={columns}
    onClick={row => openDrawer(row.id)}
  />
  <DetailDrawer open={drawerOpen} />
</PortalLayout>
```

Next: Review [Page Templates](#) for complete page compositions.